Quantum Ansatz Design With Monte Carlo Tree Search

Master Research - Final Report

Authors

Andres Lebbos Habchi Mathias Yap Vincent Hoogstraten

Emil Künstner Emir Ors

Supervisors

Vincenzo Lipardi Dr. Dennis Soemers Dr. Domenica Dibenedetto

Coordinator

Gijs Schoenmakers

January 22, 2025

Abstract

Variational Quantum Algorithms use parameterized quantum circuits to solve optimization problems, where selecting an effective circuit design, or ansatz, is crucial but challenging. Quantum Architecture Search automates this process, but the vast search space of possible circuits makes it computationally demanding. This study explores how variants of Monte Carlo Tree Search can improve Quantum Architecture Search by enhancing search efficiency and solution quality. Key techniques include introducing terminal nodes, grouping circuits based on gate types or actions, and using progressive widening to manage exploration. These variants are tested individually and in combination on quantum optimization problems, like ground-state energy estimation and variational quantum linear solvers, with findings evaluating the efficiency of Monte Carlo Tree Search variants. Although no single variant proved to be best across all problems, for each problem there exists a variant that can improve the baseline Monte Carlo Tree Search. Furthermore, progressive widening with any grouping improved ground-state energy problems, while grouping by gates proved to be most promising in variational quantum linear solvers.

1 Introduction

Variational Quantum Algorithms (VQAs) [1] are hybrid quantum-classical optimization algorithms designed to leverage both quantum and classical computational resources. They are implemented by framing problems as optimization tasks, where the solution is found by iterating over a parameterized quantum circuit (PQC), known as an ansatz. The ansatz serves as an educated guess for the solution of the problem, with tunable parameters that are optimized to minimize a cost function. The process combines quantum circuit evaluations with classical optimization techniques.

The effectiveness of VQAs is heavily contingent on the choice of the initial ansatz and designing an effective ansatz remains one of the central challenges in VQAs [1]. Different application domains often require tailored ansatz designs that account for the problem structure and the limitations of quantum devices. This is

a non-trivial, time-consuming task that requires expert knowledge. Given the critical role of the ansatz, Quantum Architecture Search (QAS) has emerged as an area of research into automatically exploring and optimizing the structure of PQC's[2] . The goal is to systematically identify the most effective PQC for a given problem by navigating the search space of quantum circuits.

Efficiently exploring the search space of quantum circuits presents a significant challenge due to its vastness. Various techniques have been employed to address this issue, including evolutionary strategies [3] [4], reinforcement learning [5], differentiable algorithms [6], and Bayesian optimization [7]. Artificial intelligence techniques may help to explore more effectively. One such AI method, Monte Carlo Tree Search (MCTS) [8, 9, 10], has been adapted from classical applications to the QAS problem [11].

This work explores the application of MCTS to QAS tasks by investigating the impact of incorporating variants, such as gate and action-based grouping [12], progressive widening within the groups, and terminal nodes on the algorithm's performance across a range of quantum optimization problems. Through experimentation, this work analyzes the effect these variants have on search efficiency, solution quality, and quantum circuit complexity.

In the context of quantum computing, MCTS has been employed for discrete optimization in the Quantum Approximate Optimization Algorithm [13], Quantum circuit transformation [14] and quantum dynamics optimization [15]. Additionally, MCTS has found utility in the related area of Neural Architecture Search [16, 17, 18].

2 Theoretical Framework

This paper builds on the research done by Lipardi et al. [11]. QAS is formally defined in Section 2.1, a general introduction to MCTS is provided in Section 2.2, and a simple overview of the existing framework is provided in Section 2.3.

2.1 Quantum Architecture Search

Quantum Architecture Search aims to find the parameterized quantum circuits (PQCs) that optimize a given objective function. Specifically, it has to determine the topology of the PQC, namely an ordered sequence of quantum gates, their respective positions, and the corresponding angle parameters [11].

2.2 Monte Carlo Tree Search

MCTS is a best-first search technique that has shown success in solving various search-based problems [8, 9, 10]. One of its key advantages is that it does not rely on domain-specific knowledge, making it suitable for creating agnostic algorithms [19, 20].

MCTS explores the search space randomly and builds a search tree based on the outcomes of previous explorations, gradually improving its estimates of promising states. The process involves four steps [21], repeated as long as there is time left. (1) In the selection step the tree is traversed from the root node downwards until a state is chosen, which has not been stored in the tree. (2) Next, in the roll-out step, moves are chosen randomly until an end state is reached. (3) Subsequently, in the expansion step one or more states encountered along its roll-out are added to the tree. (4) Finally, in the back-propagation step, the results are propagated back along the previously traversed path up to the root node [11].

Starting from only the root, MCTS aims to visit the nodes leading to the best result so far, based on the estimated probability of success (exploitation). On the other hand, it is important to further explore less visited nodes, which due to the uncertainty of the simulations might reveal to be closer to the optimum (exploration) [11].

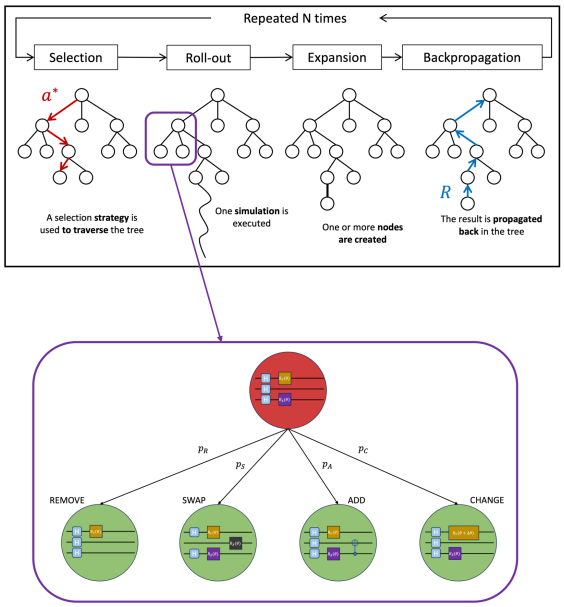

Figure 2.1: Monte Carlo Tree Search Scheme. Starting from the root node, the tree is gradually grown by executing the selection, roll-out, expansion, and backpropagation steps [22]. In the provided QAS framework, the action space is defined by sampling in a continuous set of possible quantum circuit modifications. [11]

2.3 Monte Carlo Tree Search for Quantum Architecture Search

Lipardi et al. [11] use Monte Carlo Tree Search, where each node represents a quantum circuit, and each move corresponds to a modification of that circuit. MCTS explores potential modifications by sampling from four types of actions: adding, swapping, deleting gates, or adjusting the angle parameters of existing gates. These actions are selected according to a probability distribution that evolves throughout the search, favoring additions early on and adjusting based on factors such as circuit depth.

A key aspect of this framework is the reward function, which evaluates each quantum circuit configuration. While traditional MCTS applications may focus on rewards at terminal states, in this domain, the concept of terminal states is less clear and every node can be treated as both terminal and non-terminal. This allows each visited node to be evaluated directly, without requiring a roll-out to a distant terminal node. However, roll-outs can still provide valuable information for future decisions [11].

As MCTS explores different configurations, it progressively builds the search tree, committing to the best action when a certain threshold of visits is reached at any given node. This strategy ensures that the search focuses on promising regions of the solution space without prematurely converging. To handle the vast, continuous search space, a progressive widening technique is incorporated that restricts the number of actions to a manageable subset at each step, allowing MCTS to focus on the most relevant nodes [11].

Once the search concludes, the best quantum circuits are extracted from the search tree, and their parameters are fine-tuned using a classical gradient-based optimizer. This approach enables efficient exploration of both the topology and parameters of quantum circuits, aiming to find near-optimal configurations for variational quantum algorithms [11].

3 Methodology

This section provides a detailed overview of the different MCTS variants, the research questions they address, and the hypotheses they aim to improve. These variants, individually or in combination, are applied to solve a set of QAS problems. The problem set includes estimating the ground state energies of H_2, H_2O, and LiH, as well as solving two systems of linear equations using a variational quantum linear solver (VQLS).

All variants and combinations use the best performing model parameters identified by Vincenzo Lipardi et al. [11], which include no roll-out, an upper confidence bound of 0.4, a progressive widening coefficient of 1, and a progressive widening alpha of 0.3 [21].

The performance of the MCTS variants and their combinations is evaluated across 10 independent runs, where each run is limited to 5000 quantum circuit evaluations in the MCTS, followed by a maximum of 200 iterations of gradient descent-based parameter optimization using the Adam optimizer [23], to further refine the cost of the MCTS's best greedy and overall quantum circuits.

The evaluation metrics used to assess performance are (1) a problem dependent reward and (2) the quantum circuit complexity (number of R-Gates and Cx-Gates).

The primary task of this report is to find quantum circuits which solve the problem at hand. However, in quantum circuits, if a smaller circuit solves the problem with the same or better result, the smaller one is more valuable for various reasons. Specially with regards to R-Gates and Cx-Gates, it should be noted that the less quantum gates in the quantum circuit, the better it is. On one hand, the less R-Gates the circuit has, the easier it is to optimize the circuit, because the amount of parameters over which the optimization happens depends directly from the amount of R-Gates present. On the other hand, the less Cx-Gates a circuit has, the more reliable and less noisy the results are. Therefore, if we can achieve equal or higher results after gradient descent optimization when compared to the baseline, while reducing the amount of R-Gates and Cx-Gates, then the result is less noisy, more reliable, and easier to optimize.

3.1 Terminal Nodes

Terminal nodes in the context of MCTS shown in Figure 3.1 refer to nodes in the search process where further expansion is stopped. In the usual MCTS domain of games, a reward function is defined for leaf nodes and the value of internal nodes can be inferred from their children. However, in the domain of QAS, each internal node also has a valid accurate evaluation [11]. This implies that a better quantum circuit could be found at higher levels within the tree.

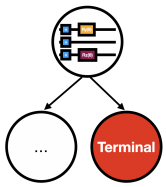

Figure 3.1: Visual representation of terminal nodes.

1 ***How can terminal nodes improve Monte Carlo Tree Search for Quantum Architecture Search?***

Hypothesis: Terminal nodes are expected to improve MCTS by identifying sufficiently optimized solutions early in the search. By limiting the search depth and tree size, unnecessary exploration of suboptimal paths could be decreased, thereby improving the search process. Additionally, terminal nodes could serve as checkpoints, highlighting promising circuit designs that could be further refined. However, this increases the risk of overlooking better solutions in unexplored parts of the tree by prematurely stopping node expansions.

3.2 Grouping

Grouping techniques in decision algorithms have been widely explored. In games like Shogi, moves are categorized based on simple features to allocate computational resources more effectively [24]. In Go, similar methods group moves into categories such as "Good" and "Bad" for collective evaluation [25].

In standard MCTS, nodes represent individual states while edges represent moves. Group nodes aggregate moves that share common features, reducing the need to evaluate each move separately. By focusing on groups, MCTS can prioritize broader categories in the early stages, allowing more efficient exploration before refining individual options. This flexible structure significantly enhances search efficiency, particularly when dealing with complex decision spaces [12].

3.2.1 Group by Gates

Group by gates groups children circuits by gate type, as illustrated in Figure 3.2. By defining a finite amount of children for each gate type with a fixed parameter, the continuous, infinite branching factor is reduced to a discrete, finite one. Furthermore, a deletion group is added, giving the possibility to remove gates with non-efficient parameter values.

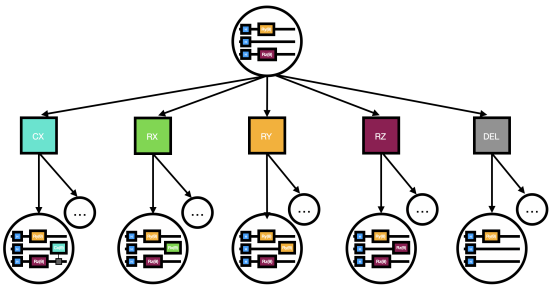

Figure 3.2: Visual representation of group by gates where each square represents a gate type group, including the deletion group in grey.

2 *How can grouping by gates improve Monte Carlo Tree Search for Quantum Architecture Search?*

Hypothesis: Selecting the gate type first, before specifying parameters, might lead to better exploration of the solution space. When adding a quantum gate, this approach emphasizes the selection of the gate type over the parameter and the qubits, which might be the most critical factor. Moreover, enabling the deletion of gates is needed to maintain flexibility; otherwise, a premature random parameter could lock the circuit into a suboptimal configuration. However, this grouping may inherently favor gate heavy solutions, such as circuits containing an increased number of Cx gates. While such gates are essential for quantum operations, they are also known to amplify noise.

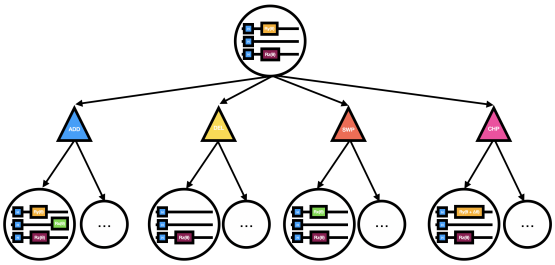

Figure 3.3: Visual representation of group by actions where each triangle represents an action.

3.2.2 Group by Actions

Group by actions categorizes circuit modifications into high-level actions as illustrated in Figure 3.3. The actions that can be performed are the following: (1) Add, which adds a gate to the circuit (2) delete, which removes a gate from the circuit (3) swap, which exchanges a gate on the existing circuit with a new random one and (4) change, which changes the parameter of a parameterized gate. Each group will include all the circuits that result from applying its corresponding action with random parameters for new gates.

3 *How can grouping by actions improve Monte Carlo Tree Search for Quantum Architecture Search?*

Hypothesis: The main benefit of the grouping by actions is that it can choose for each circuit what type of modifications is valuable: adding a gate, modifying a parameter of a gate, swapping a gate or deleting a gate. However, a potential drawback of these groupings is the high branching factor. This can lead to inefficiencies, as some actions may be irrelevant given the circuit's current state. Despite being unlikely to yield improvements, these actions are still expanded, consuming significant evaluation resources on unpromising or even detrimental circuit modifications.

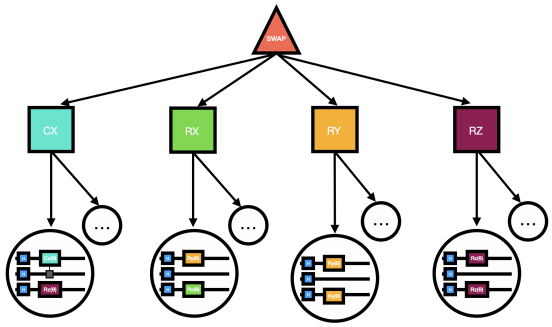

Figure 3.4: Visual representation of the subgroups of swap, where the triangle represents the swap action and the squares represent the gate types.

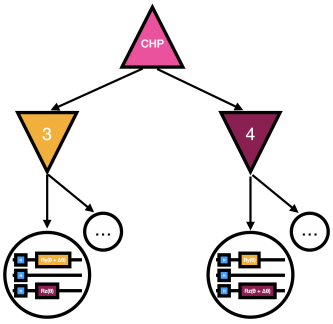

Figure 3.5: Visual representation of the subgroups of change, where the triangle pointing up represents the change action and the triangles pointing down represent the gate positions in the circuit.

3.2.3 All Groupings

All groupings further expands group by actions by creating subgroups in each action: the subgroups for (1) add and (2) swap are for each gate type, similar to group by gates, while the subgroups for (3) change are for each existing gate. The subgroups of add follow the same structure as group by gates without the delete group. The subgroups of swaps are grouped by the gate type of the new gate as shown in 3.4. The subgroups of change are grouped by all existing parametrized gates as shown in 3.5, while maintaining progressive widening for the infinite branching factor.

4 *How can all groupings improve Monte Carlo Tree Search for Quantum Architecture Search?*

Hypothesis: All groupings further expands the idea of grouping, adding the possibility to prioritize certain subgroups in the remaining actions. However considering the change action contains of more subgroups, more circuits must be evaluated. This increases the computation time, while it is not guaranteed that an arbitrary change of a parameterized gate changes the output of a quantum circuit.

3.2.4 Progressive Widening

Despite progressive widening [26] already being used for the change action, the remaining groupings still have a high, finite number of children. By also introducing progressive widening in such groups, the number of initial evaluations can be limited to the number of groups.

5 *How can progressive widening improve Monte Carlo Tree Search for Quantum Architecture Search?*

Hypothesis: This approach ensures that initially, only a small subset of promising actions is explored, avoiding computational overhead, while gradually incorporating more expansions as more information about the circuit becomes available.

3.3 Combinations

In addition to the variants themselves, performance across different combinations is measured, potentially enhancing the performance of MCTS for the QAS problems. Combining grouping by gates with grouping by actions keeps the structure of the grouping by actions, while expanding the add action with every gate type. A variant

combination including terminal nodes contains an identical terminal child for each non terminal circuit while a variant combination including finite progressive widening, only affects grouping variants.

6 ***What is the effect of each of these variant combinations on Quantum Architecture Search problems?***

Hypothesis: Each variant targets different aspects of MCTS and aims to improve specific components of QAS. For example, grouping affects the search tree by changing the selection process from an infinite continuous branching factor to a finite discrete one, while terminal nodes limits the infinite depth of the tree. Due to this fact, combining variants might further improve the performance of the QAS problems and help limitations that the other might encounter.

4 Results

Based on an initial exploration and experimentation with the MCTS model developed by Vincenzo Lipardi et al. [11], the best model was selected as a baseline. This baseline employs a probabilistic expansion strategy, with actions distributed as follows: 50% addition, 20% modification, 20% swapping, and 10% deletion of nodes. A minor adjustment was made by removing the committing mechanism, ensuring that the root node remained fixed throughout the search process. A detailed comparison of these modified standard versions is provided in Figure A.1.

Each variant, excluding progressive widening for finite groupings, is evaluated for every QAS problem where the ground-state energy estimation problem for LiH as shown in Figure 4.1 which highlights significant differences between the variants within the MCTS process. Although some variants showed minor improvements over the baseline model in the ground-state energy estimation problem H_2O and the systems of linear equation, VQLS 0, none of them significantly

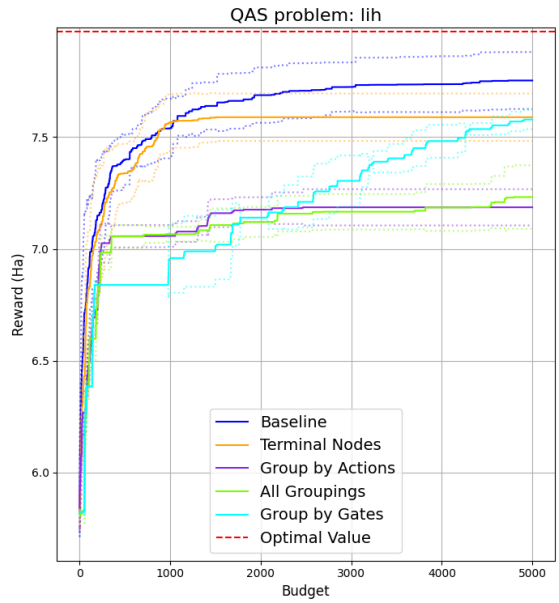

Figure 4.1: Average reward convergence over 10 runs for the baseline model and its variants, with dotted lines representing the rewards' standard deviations, plotted against the number of evaluations used in MCTS for the ground-state energy estimation of LiH.

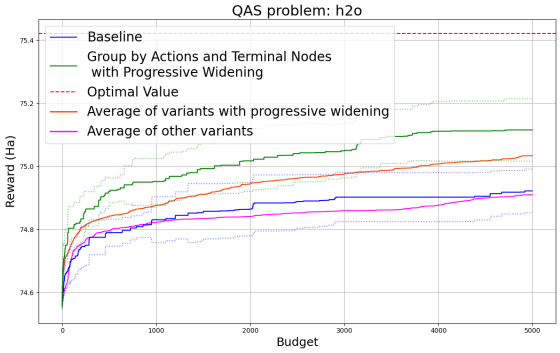

Figure 4.2: Average reward convergence over 10 runs for the baseline model, the best variant combination; Group by Actions and Terminal Nodes with progressive widening, and the average reward over variants with and without progressive widening, with dotted lines representing the rewards' standard deviations, plotted against the number of evaluations used in MCTS for the ground-state energy estimation of H_2O.

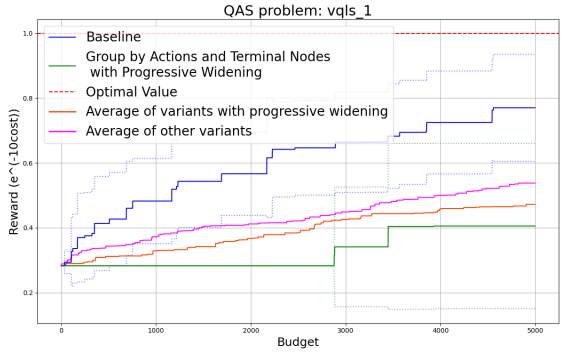

Figure 4.3: Average reward convergence over 10 runs for the baseline model, the best variant combination; Group by Actions and Terminal Nodes with progressive widening, and the average reward over variants with and without progressive widening, with dotted lines representing the rewards' standard deviations, plotted against the number of evaluations used in MCTS for the problem of linear equations VQLS 1.

improved the MCTS performance for the other QAS problems, as shown in Figures A.2, A.3, A.4, and A.5.

The variants, including progressive widening for finite groupings, showed significant differences from the variants without it, as shown in Figures A.6, A.7, and A.8. Progressive widening significantly improved the ground-state energy estimation problems, where the average MCTS performance on H_2O across different variants as shown in Figure 4.2. However, some QAS problems seem to prefer variants without progressive widening as shown in Figure 4.3.

Figure 4.4 shows the results after gradient descent optimization for eight of the variants across three problems. B contains results on all problems for all tested variants. For all ground-energy state problems, the variant that combines grouping by action with progressive widening outperformed all others, as shown in Figures B.1, B.2, and B.3. This variant achieved optimal solutions in at least one of the ten runs for all three problems and was the only variant to find an optimal solution for the H_2O problem.

Additionally, Figure C.3 highlights that progressive widening improved the performance across all variants on the three ground-state energy problems, with LiH showing the clearest improvements shown in Figure 4.5. However, in the VQLS problems, the impact of progressive widening varied. In VQLS 0, its inclusion had little effect, with scores similar to those obtained without it as shown in Figure B.4. In VQLS 1, progressive widening reduced performance across all variants as shown in Figure 4.5.

While grouping by gates did not improve the performance on the ground-state energy problems, Figure 4.6 shows that it leads to improved results across both VQLS problems. For them, the individual variants that obtained the best results both included grouping by gates as show in Figures B.4 and B.5. Neither grouping by actions nor terminal nodes consistently improved performance across either category of problems; their effects varied, sometimes enhancing and sometimes diminishing performance as shown in Figures C.1 and C.4). However, the best performing variant for each of the problems incor-

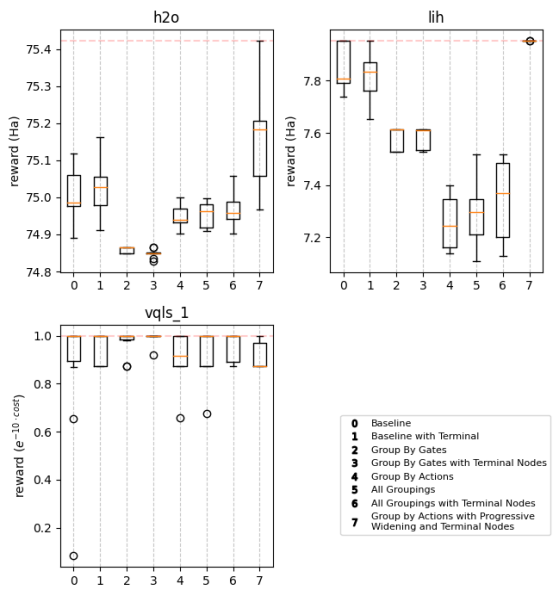

Figure 4.4: Box plot of the highest reward values achieved after gradient descent parameter optimization for each of the 10 runs, for the best amongst the greedy and best circuits identified during the MCTS search. Results are presented for the baseline, four individual variants, and the two best variant combinations on the problems H_2O, LiH, and VQLS 1. The dotted red lines indicate the optimum reward value for each problem.

porated terminal nodes, and for all problems except VQLS 1 it also included grouping by actions as shown in Figure B).

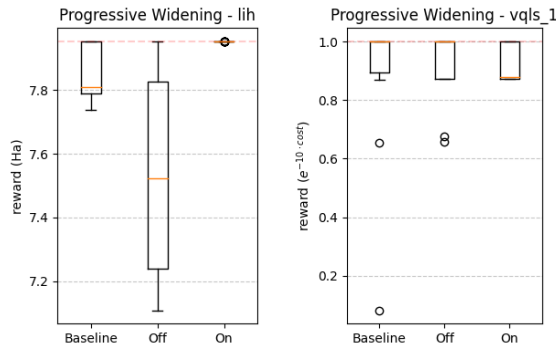

Figure 4.5: Box plot of the highest reward values achieved after gradient descent parameter optimization for each of the 10 runs for all variants that include progressive widening versus all variants that do not. The baseline variant's reward is plotted over its 10 runs. Plotted for VQLS 0 and VQLS 1 problems.

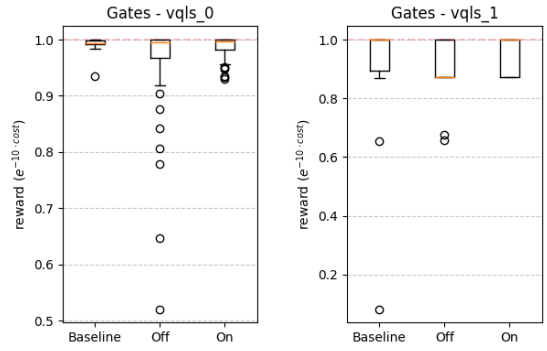

Figure 4.6: Box plot of the highest reward values achieved after gradient descent parameter optimization for each of the 10 runs for all variants that include gates versus all variants that do not. The baseline variant's reward is plotted over its 10 runs. Plotted for VQLS 0 and VQLS 1 problems.

Table 1 shows that the variant composed by

Problem	H_2	H_2O	LiH	VQLS 1
Qubits	4	8	10	4
Baseline				
H-Gates	1	8	1	4
R-Gates	6	**11**	8	**2**
Cx-Gates	0	**1**	0	**2**
Before	1.117	75.080	7.948	0.951
After	1.117	**75.117**	7.953	1.000
All Groupings, Terminal Nodes				
H-Gates	2	5	7	4
R-Gates	2	**3**	2	**2**
Cx-Gates	0	**0**	0	**0**
Before	1.114	74.994	7.953	0.966
After	1.117	75.057	7.953	1.000
Group by Actions, Terminal Node, Progressive Widening				
H-Gates	1	2	0	4
R-Gates	4	**6**	2	**3**
Cx-Gates	1	**0**	0	**0**
Before	1.116	75.268	7.953	0.892
After	1.117	**75.422**	7.953	1.000

Table 1: Qubit and gate composition, along with rewards before and after gradient decent parameter optimization, for the best optimized circuit among 10 trials for each QAS problem: H_2, H_2O, LiH, and VQLS 1. Shown for the baseline model, the variant combination of all groupings with terminal nodes, and variant combination of group by actions with terminal nodes and progressive widening.

group by actions, terminal nodes, and progressive widening achieves better results in quantum chemistry problems like H_2O, while reducing both the amount of R-Gates and Cx-Gates. Additionally, variants without progressive widening, like the one with all groupings and terminal nodes, achieves better results than the baseline. This is because the post optimization values are equal or slightly lower than the baseline, while the amount of R-Gates and Cx-Gates used is significantly lower.

5 Discussion

This section discusses the impact of dynamic progressive widening within the groupings, the impact of grouping on the structure and complexity of the most optimal circuit found by the variants with any grouping variant and near-optimal solutions with MCTS and gradient descent optimization.

5.1 Impact of progressive widening on various QAS problems

The varying effectiveness of MCTS modifications across different problem types, such as VQLS and quantum chemistry problems like H_2, H_2O, and LiH, might stem from the distinct structural and symmetry properties of these domains. In quantum chemistry problems, the optimal ansatz often exhibits a high degree of symmetry due to the uniformity of interactions within molecular systems. Progressive widening enhances performance here by directing the search towards specific gate types that align with these symmetric patterns, avoiding unnecessary exploration of qubit specific variations. Conversely, VQLS problems often lack such inherent symmetry, with optimal circuits depending more on precise qubit interactions and unique configurations as shown in Figure 4.5. In these cases, progressive widening may oversimplify the search by overly restricting qubit placement exploration, reducing its effectiveness. This difference highlights the importance of tailoring MCTS modifications

to the problems' structural characteristics to balance exploration and exploitation effectively.

Progressive widening within the finite groupings results in searching deeper trees instead of broader ones, which would prioritize a search on the gate types over the qubits each gate affects. Without progressive widening, each grouping variant instantly adds all children in the tree. If the circuit has n qubits and m quantum gates, then the add section would have $n(n-1)$ children under the Cx-Gates group and n children under each R-Gate group for a total of $n(n-1)+3n = n^2+2n$ children, the swap section would have m children under each type of gate for a total of $4m$ children, and the delete section would have m children. Therefore, the finite branching factor for these categories together can be approximated to $n^2+2n+5m = O(n^2+m)$. Due to this being a very large number of children, this results in broader trees that search not only for all the quantum gate types but also for all the qubit combinations they can affect. Depending on the QAS problem being solved, this could be beneficial or not.

Every QAS problem is different and some seem to prefer deeper trees over broader ones. For example, progressive widening on the finite groupings outperform all variants in quantum chemistry problems like H_2, H_2O, and LiH but is not that great in other quantum problem like VQLS 0 and VQLS 1 as shown in Figure 4.5. This might be due to the fact that the quantum chemistry problems are more symmetric around the qubits when compared to the others. Furthermore, the type of quantum gates could be far more important than the qubit placement of the gates in these types of symmetric problems when compared to other problems and this results are better explored through deeper trees. Instead of searching through all qubit combinations for a gate, the finite progressive widening variants prioritize the type of gates over the qubits they affect through a narrower search which is highly beneficial on symmetric problems like the ones in quantum chemistry.

5.2 Impact of Groupings on Quantum Circuit Complexity

Tables 2 and 3 show that the variants composed by any combination of groupings tend to achieve better results while also reducing both the amount of R-Gates and Cx-Gates when compared to variants without any groupings. The variants without groupings lean towards higher number of R-Gates and Cx-Gates. By using progressive widening on an entire action instead of limiting the action by more specific groupings like gates as shown in Figure 4.6, these variants explore all combinations of gates instead of potentially adding unnecessary gates that do not affect negatively the outcome. This guides MCTS into making sure the smaller circuits with different gate patterns are rigorously explored before the bigger circuits resulting in circuits with similar or better results but far less R-Gates and Cx-Gates.

5.3 Near-Optimal Solutions with MCTS and Gradient Optimization

MCTS variants proved to be effective in navigating the vast search space of quantum circuit designs, often reaching solutions with scores that are near-optimal as shown in Figure 4.2. This performance demonstrates the algorithm's capability to identify promising regions within the search space efficiently. However, due to its probabilistic nature and reliance on exploration versus exploitation trade-offs, MCTS alone occasionally fell short of achieving perfect optimization. When combined with gradient-based optimization techniques, which fine-tune circuit parameters with precision, the solutions were further refined to achieve optimal or near-perfect scores as shown in Figure 4.4. This complementary approach highlights the strength of MCTS in identifying robust initial solutions and the precision of gradient methods in achieving exact optimization.

6 Conclusions

This project investigated several improvements to MCTS for QAS with the goal of improving upon the baseline MCTS proposed by Lipardi et al. [11]. Our approach disabled node commitment during the search and utilized a variant where deterministic stopping was deactivated, leading to a probabilistic stopping. This variant served as the baseline for evaluation of the proposed modifications.

The main objective was to explore how different strategies and configurations could enhance the baseline performance in the context of QAS. This section summarizes the insights gained from each research question.

1. **How can terminal nodes improve Monte Carlo Tree Search for Quantum Architecture Search?**

   Incorporating terminal nodes into the MCTS did not perform better when applied to most variants and across most problems. Interestingly, although the addition of terminal nodes often did not lead to any improvement, both the best variant combination for the ground-state energy problems and the best variant combinations for the VQLS problems included terminal nodes.

2. **How can grouping by gates improve Monte Carlo Tree Search for Quantum Architecture Search?**

   Grouping by gates worsened performance for quantum chemistry problems, while it led to an increased performance for solving linear systems across most of the variants, including the highest performing ones.

3. **How can grouping by action improve Monte Carlo Tree Search for Quantum Architecture Search?**

   Incorporating grouping by actions into the MCTS overall did not perform better when applied to most variants, leading to a decrease in performance for three of the problems overall. As with terminal nodes, the best variant combination for the ground-state energy problems and the best variant combination for the VQLS 0 problem, included grouping by actions.

4. **How can all groupings improve Monte Carlo Tree Search for Quantum Architecture Search?**

   Combining multiple grouping strategies (e.g action, gates, change, and swap) showed mixed results. This approach matched or exceeded the baseline on three of five problems: H_2, H_2O, and VQLS 0. On the other hand, it performed significantly worse on the other two problems: LiH and VQLS 1. These results indicate that while combinations of groupings can sometimes improve performance, they also introduce inconsistencies.

5. **How can progressive widening improve Monte Carlo Tree Search for Quantum Architecture Search?**

   Progressive widening significantly improved performance across most variants on the ground-state energy problems. It elevated all approaches to at least baseline-level performance, with some configurations surpassing it. Notably, combining grouping by action, terminal nodes, and progressive widening performed best, outperforming the baseline on four out of five problems, excluding VQLS 1.

6. **What is the effect of each of these variant combinations on Quantum Architecture Search problems?**

   The findings suggest that while individual modifications may not reliably outperform the baseline, certain combinations (e.g., progressive widening with specific grouping strategies) show promise in enhancing MCTS performance for QAS. The Version with Group by actions and terminal nodes together with progressive widening showed overall the most promising results.

7 Future Work

The following sections define various different ways to further expand on the research done in this project.

7.1 n-grams

An n-gram, in MCTS for QAS, is a sequence of n particular gates used to capture the frequency at which these sequences show up in the search tree. Particularly, n-grams are an ideal target in MCTS because they don't require prior specific knowledge and can be constructed dynamically as the search tree is explored. Furthermore, n-grams have been used to enhance MCTS in particular domains like board game AI [27].

The hypothesis is that certain sequences of quantum gates may form efficient patterns for the current iteration of MCTS. The idea involves training n-grams as the search tree is explored by leveraging the probability distributions encountered in the back propagation process while keeping track of how effective those sequences are at solving the design problem at hand. These n-grams would then be used to make well informed decisions during the selection phase of MCTS. By doing so, the objective is to speed up MCTS by selecting more promising nodes that have better odds at providing successful results when probed. Investigating the impact of n-grams on search performance and exploring adaptive strategies for their implementation are promising directions to improve the overall effectiveness of this algorithm.

7.2 Nested MCTS

Nested MCTS in QAS allows MCTS to work recursively, exploring each part of the search space deeper. It helps balance exploring the whole search space and focusing on specific promising areas, leading to better circuit optimization. Nested MCTS can also evaluate rewards more precisely and refine circuit structures step by step. Although it requires more computation and careful parameter tuning, it has great potential to improve search efficiency and the quality of solutions, making it another promising direction worth exploring.[28]

7.3 Dynamic Progressive Widening

To improve progressive widening for problems like VQLS 1, where current variants did not perform well, future research can try adjusting the speed of expansion dynamically. Instead of a fixed rate, the algorithm could add new actions based on metrics like node visits or reward changes. For example, if rewards are low, it could expand faster to explore new possibilities. If rewards are high, it could slow down to focus on refining promising areas. Future research can use adaptive strategies that vary expansion rates in different parts of the search tree. Faster expansion can help in regions with steep reward changes, while slower expansion can avoid wasting resources in less promising areas.[29]

7.4 Multi-Armed Bandit on Variant Selection

Although some of the variants discussed in this report consistently show good results across most of the tested problems, there is not a clear variant which is ideal for every category of QAS problems. Additionally, while a variant of MCTS can fail at all the explored problems in this report, there could be categories of quantum problems where that variant could outperform every other one. Future research can explore a Contextual Multi-Armed Bandit Problem approach to discover, given a category of quantum problems, which variant could be used best to solve those problem. In this approach, the suggestion is to use the variants as arms, quantum problems as time steps, and category specific properties like symmetry across quantum bits as the context.

7.5 R-Gates and Cx-Gates Circuit Prioritization

MCTS can be adapted to recognize that quantum circuits prefer smaller circuits with similar or better results. The selection or evaluation process can be adapted to take into the account the number of R-Gates and Cx-Gates in order to guide the search algorithm into prioritizing smaller quantum circuits that can achieve similar or better results. Although the variants discussed in this report consistently show smaller circuits than the baseline as shown in Table , it is still possible than even smaller circuits can solve the quantum problem and MCTS can be trained to prioritize them.

References

[1] M. Cerezo et al. "Variational quantum algorithms". In: *Nature Reviews Physics* 3.9 (2021), pp. 625–644. DOI: 10.1038/s42254-021-00348-9.

[2] Darya Martyniuk et al. "Quantum Architecture Search: A Survey". In: (2023).

[3] Arthur G Rattew et al. "A domain-agnostic, noise-resistant, hardware-efficient evolutionary variational quantum eigensolver". In: *arXiv preprint arXiv:1910.09694* (2019).

[4] Hongxiang Wang et al. "Quantumnas: Noise-adaptive search for robust quantum circuits". In: *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 692–708.

[5] K. A. McKiernan et al. "Automated Quantum Programming via Reinforcement Learning for Combinatorial Optimization". In: *arXiv preprint arXiv:1908.08054* (2019). URL: https://arxiv.org/abs/1908.08054.

[6] Sheng-Xin Zhang et al. "Differentiable quantum architecture search". In: *Quantum Science and Technology* 7.4 (2022), p. 045023.

[7] M. Pirhooshyaran and T. Terlaky. "Quantum Circuit Design Search". In: *Quantum Machine Intelligence* 3 (2021), pp. 1–14.

[8] Cameron B. Browne et al. "A Survey of Monte Carlo Tree Search Methods". In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (2012), pp. 1–43. DOI: 10.1109/TCIAIG.2012.2186810.

[9] Maciej Wiechowski et al. "Monte Carlo Tree Search: a review of recent modifications and applications". In: *Artif. Intell. Rev.* 56.3 (July 2022), 2497–2562. ISSN: 0269-2821. DOI: 10.1007/s10462-022-10228-y. URL: https://doi.org/10.1007/s10462-022-10228-y.

[10] David Silver et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play". In: *Science* 362.6419 (2018), pp. 1140–1144. DOI: 10.1126/science.aar6404. eprint: https://www.science.org/doi/pdf/10.1126/science.aar6404. URL: https://www.science.org/doi/abs/10.1126/science.aar6404.

[11] Vincenzo Lipardi et al. "Quantum Circuit Design through Monte Carlo Tree Search". In: *unpublished* (2024). Unpublished manuscript.

[12] J.-T Saito et al. "Grouping Nodes for Monte-Carlo Tree Search". In: Jan. 2007.

[13] Jiahao Yao et al. "Monte Carlo Tree Search based Hybrid Optimization of Variational Quantum Circuits". In: *3rd Annual Conference on Mathematical and Scientific Machine Learning (MSML)*. 2022.

[14] Xiangzhen Zhou, Yuan Feng, and Sanjiang Li. "A Monte Carlo Tree Search Framework for Quantum Circuit Transformation". In: *arXiv preprint arXiv:2008.09331v2* (2020). URL: http://arxiv.org/abs/2008.09331v2.

[15] Mogens Dalgaard et al. "Global Optimization of Quantum Dynamics with AlphaZero Deep Exploration". In: *npj Quantum Information* 6.1 (2020). DOI: 10.1038/s41534-019-0241-0. URL: https://doi.org/10.1038/s41534-019-0241-0.

[16] Renato Negrinho and Geoff Gordon. "Deeparchitect: Automatically Designing and Training Deep Architectures". In: *Statistics* 1050 (2017), p. 28.

[17] Linnan Wang et al. "AlphaX: Exploring Neural Architectures with Deep Neural Networks and Monte Carlo Tree Search". In: *arXiv preprint arXiv:1805.07440* (2018). URL: https://arxiv.org/abs/1805.07440.

[18] Huan Zhao, Lanning Wei, and Quanming Yao. "Simplifying Architecture Search for Graph Neural Network". In: *arXiv preprint arXiv:2008.11652* (2020). URL: https://arxiv.org/abs/2008.11652.

[19] Hilmar Finnsson. "Generalized Monte-Carlo tree search extensions for general game playing". In: *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*. AAAI'12. Toronto, Ontario, Canada: AAAI Press, 2012, 1550–1556.

[20] Chiara F. Sironi et al. "Self-adaptive MCTS for General Video Game Playing". In: *EvoApplications*. 2018. URL: https://api.semanticscholar.org/CorpusID:3903926.

[21] Guillaume M. J-b. Chaslot et al. "Progressive Strategies for Monte-Carlo Tree Search". English. In: *New Mathematics and Natural Computation* 4.3 (Nov. 2008), pp. 343–357. ISSN: 1793-0057. DOI: 10.1142/S1793005708001094.

[22] Mark H. M. Winands. "Monte-Carlo Tree Search". In: *Encyclopedia of Computer Graphics and Games*. Ed. by Newton Lee. Cham: Springer International Publishing, 2015, pp. 1–6. ISBN: 978-3-319-08234-9. DOI: 10.1007/978-3-319-08234-9_12-1. URL: https://doi.org/10.1007/978-3-319-08234-9_12-1.

[23] Diederik P Kingma. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[24] Yoshimasa Tsuruoka, Daisaku Yokoyama, and Takashi Chikayama. "Game-Tree Search Algorithm Based On Realization Probability". In: *ICGA Journal* 25 (Sept. 2002), pp. 145–152. DOI: 10.3233/ICG-2002-25304.

[25] Guillaume Chaslot et al. "Monte-Carlo Strategies for Computer Go". In: *Proceedings of the 18th BeNeLux Conference on Artificial Intelligence* (Jan. 2006).

[26] R. Coulom. "Computing Elo Ratings of Move Patterns in the Game of Go". In: *Proceedings of the Computers Games Workshop 2007 (CGW 2007)*. Ed. by H.J. van den Herik et al. 2007, pp. 113–124.

[27] C.J.R.H. Laschet. "Selection and Play-out Enhancements in MCTS for Tron". In: *Maastricht University* (June 2012).

[28] Tristan Cazenave. "Nested monte-carlo expression discovery". In: *ECAI 2010*. IOS Press, 2010, pp. 1057–1058.

[29] Adrien Couetoux and Hassen Doghmen. "Adding double progressive widening to upper confidence trees to cope with uncertainty in planning problems". In: *The 9th European Workshop on Reinforcement Learning (EWRL-9)*. 2011.

A Line Graphs

This section shows all the line plots comparing the reward convergence over 10 runs for different variants and variant combinations, with dotted lines representing the reward's standard deviations, plotted against the number of evaluations in MCTS for the QAS problems; H_2, H_2O, LiH, VQLS 0 and VQLS 1.

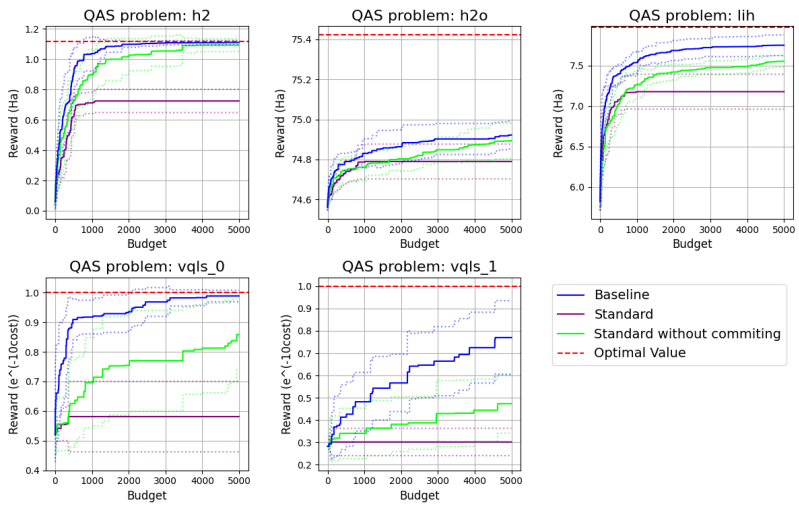

Figure A.1: Standard variants

In A.1 the convergence of the three standard variants is plotted. This work chose the best of those versions, without committing and with probabilistic stopping with a chance distribution of 50% adding a gate, 10% deleting a gate, 20% swapping a gate and 20% changing a parameter of a gate, as baseline for this and all the other plots.

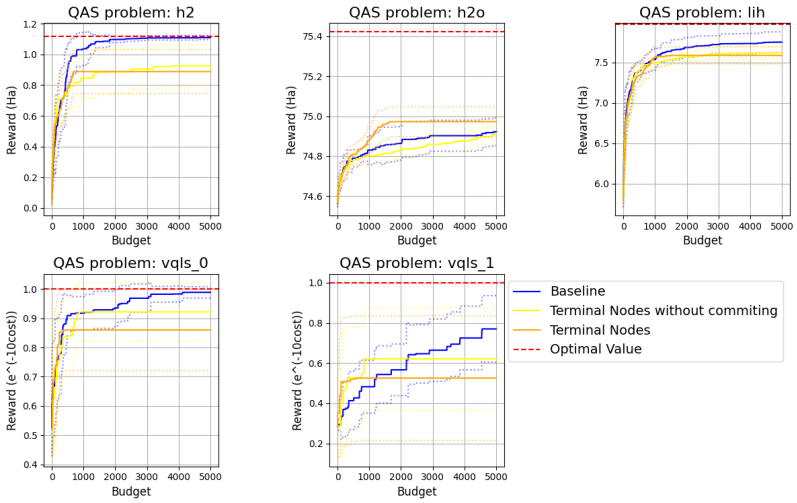

Figure A.2: Terminal variants

In A.2 the clean terminal variants are plotted against the baseline. Those variants as well as the standard variants seen in plot A.1 were already implemented in the work by Vincenzo Lipardi et al. [11]

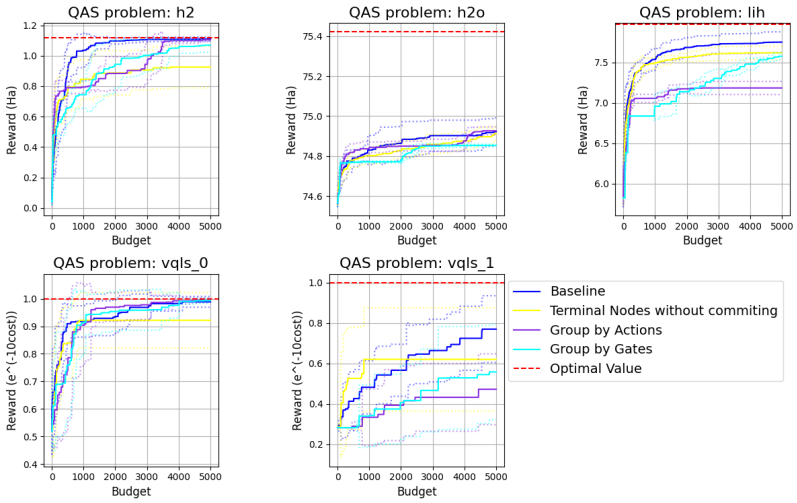

Figure A.3: Single variants

In A.3 we verify our initial hypotheses we formulated in our Project Plan and compare the pure variants of Terminal Nodes, Group by Action and Group by Gates against our baseline.

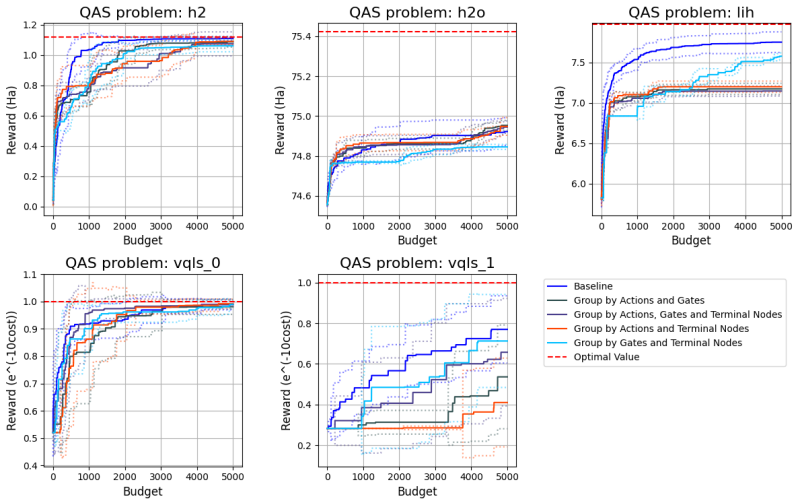

Figure A.4: Variant combinations

In A.4 we plot the variant combinations derived from our Project Plan combining Terminal Nodes, Group by Action and Group by Gates in every possible way.

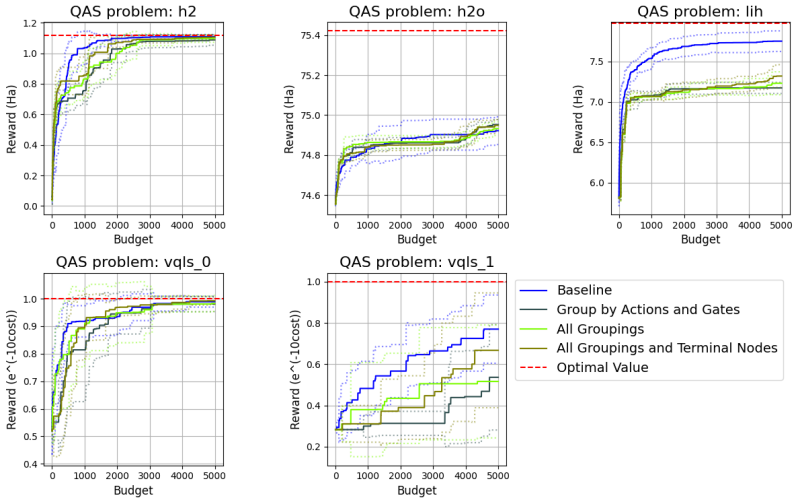

Figure A.5: All variant combinations

In A.5 we now plot all groupings adding group by swap and group by change too.

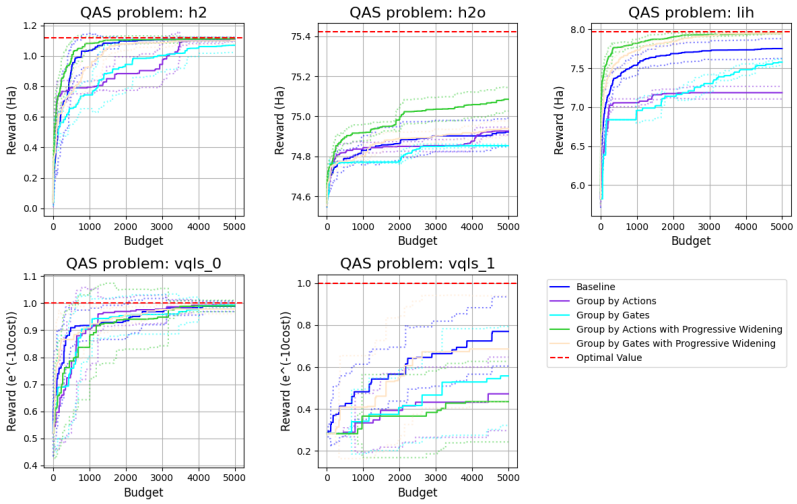

Figure A.6: Single variants with progressive widening

In A.6 we demonstrate the effect of progressive widening by comparing Group by Actions and Group by Gates in their normal forms against the enhanced forms of progressive widening.

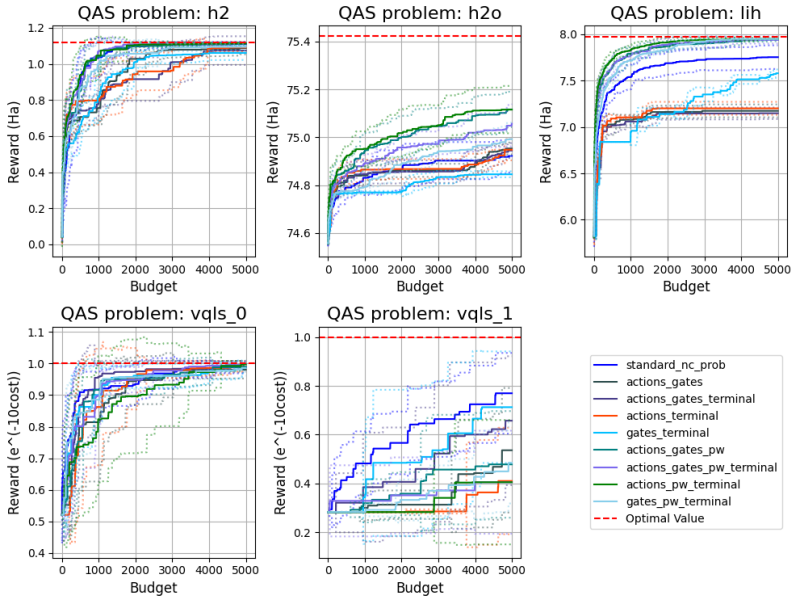

Figure A.7: Variant combinations with progressive widening

In A.7 we add progressive widening to the variants plotted in A.4.

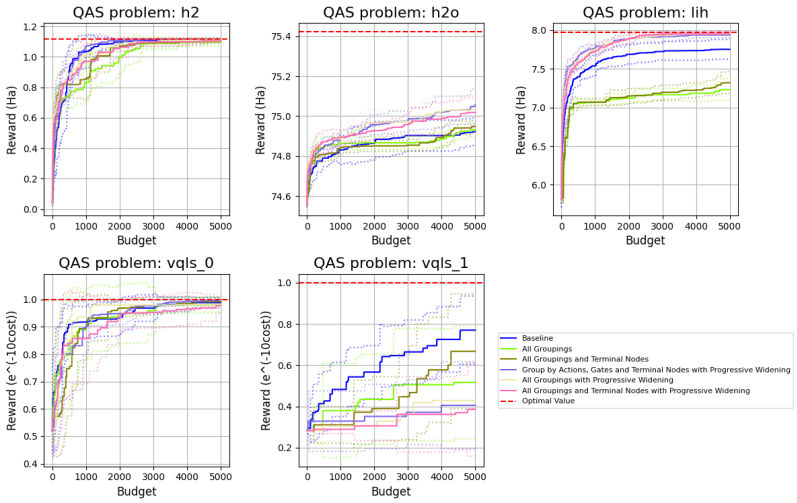

Figure A.8: All variant combinations with progressive widening

In A.8 the variants not yet shown in the previous plots are plotted. Both in their versions with and without progressive widening.

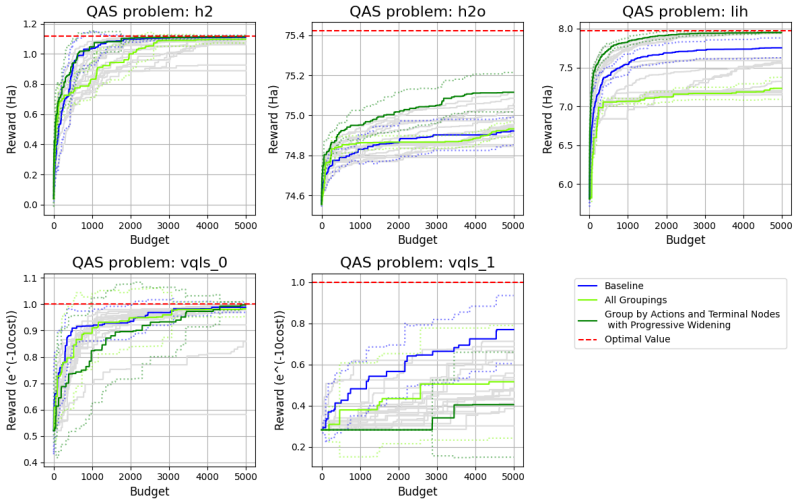

Figure A.9: All and selected variants

In A.9 we summarize the previous plot by showing the baseline, a combination with all groupings and the best variant plotted against each other. To show some proportions all not mentioned variants are plotted in grey.

B Variant Results Boxplots

The following box plots, plot the highest reward values achieved after gradient descent parameter optimization for each of the 10 runs, for either the greedy and best circuits identified during the MCTS search. Results are presented for various variants and variant combinations on the QAS problems; H_2, H_2O, LiH, VQLS 0 and VQLS 1, where the dotted red lines indicates the optimum reward value for each problem.

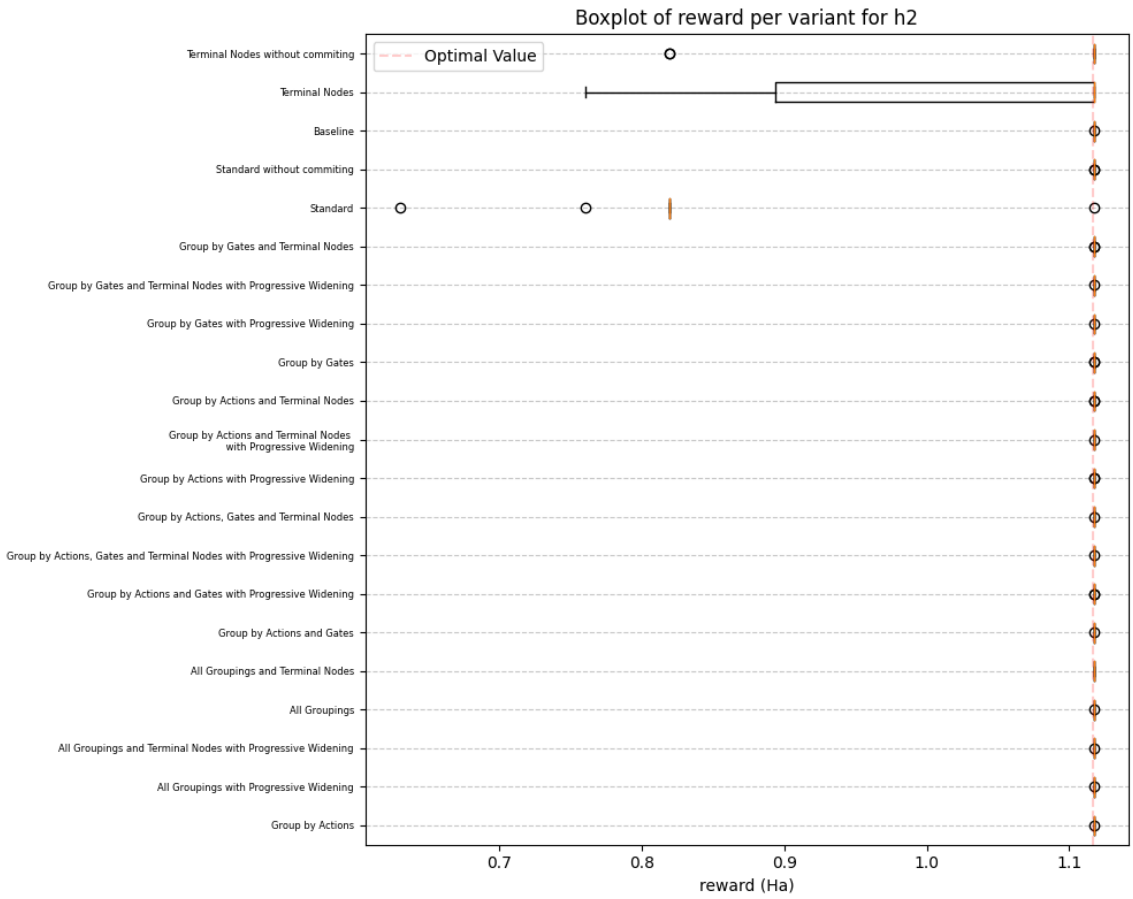

Figure B.1: All variants for problem H_2

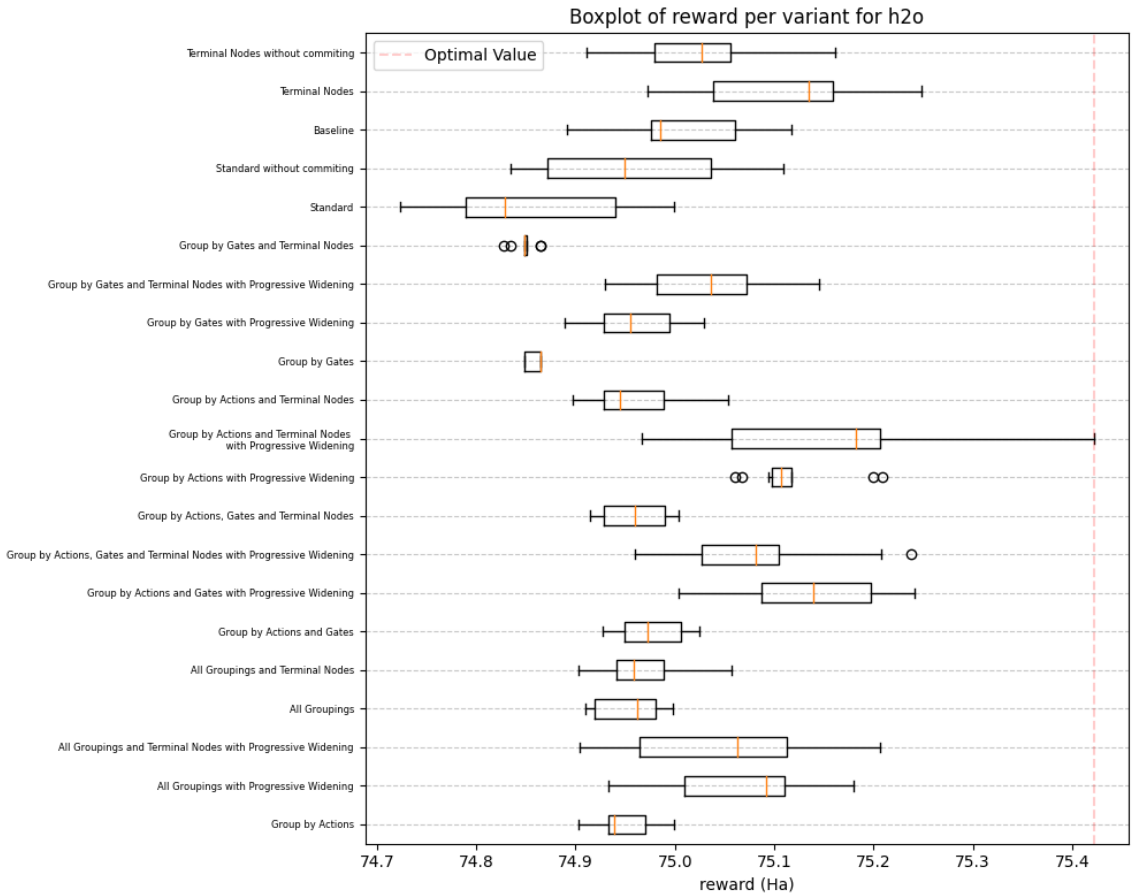

Figure B.2: All variants for problem H_2O

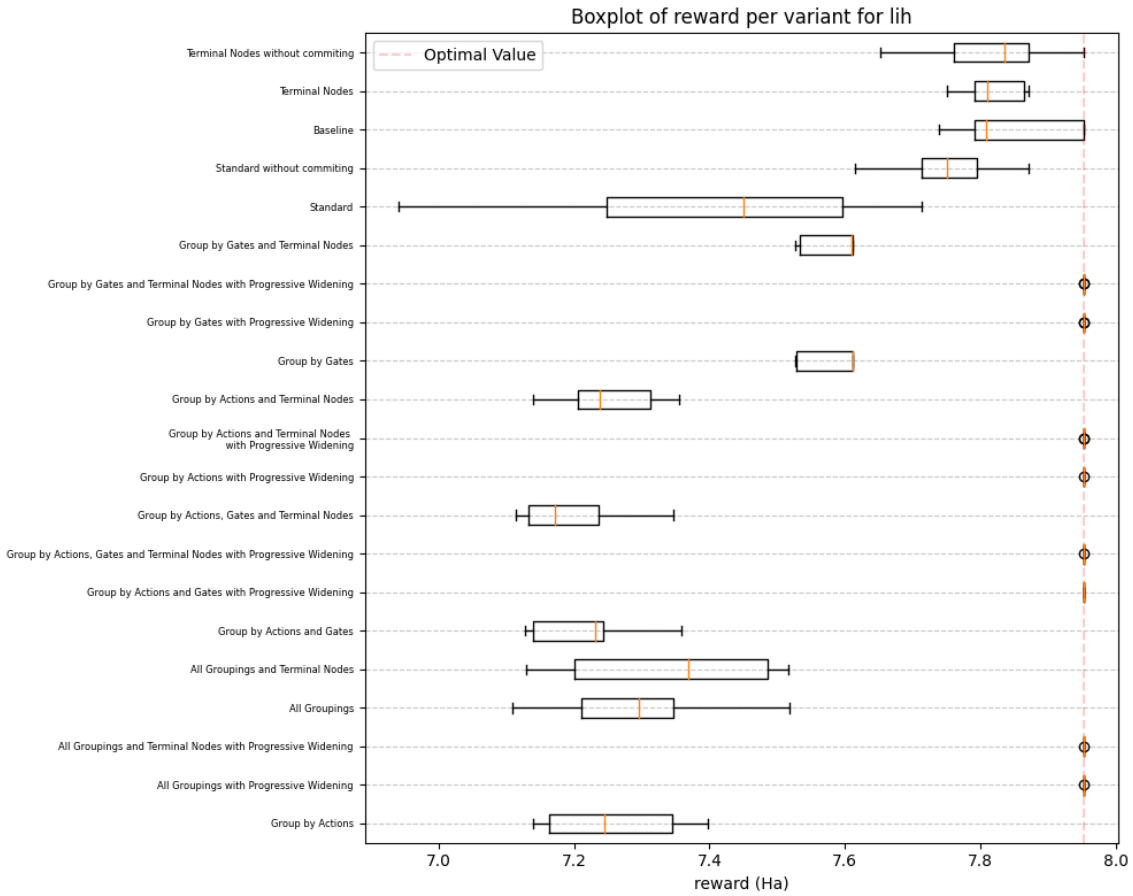

Figure B.3: All variants for problem LiH

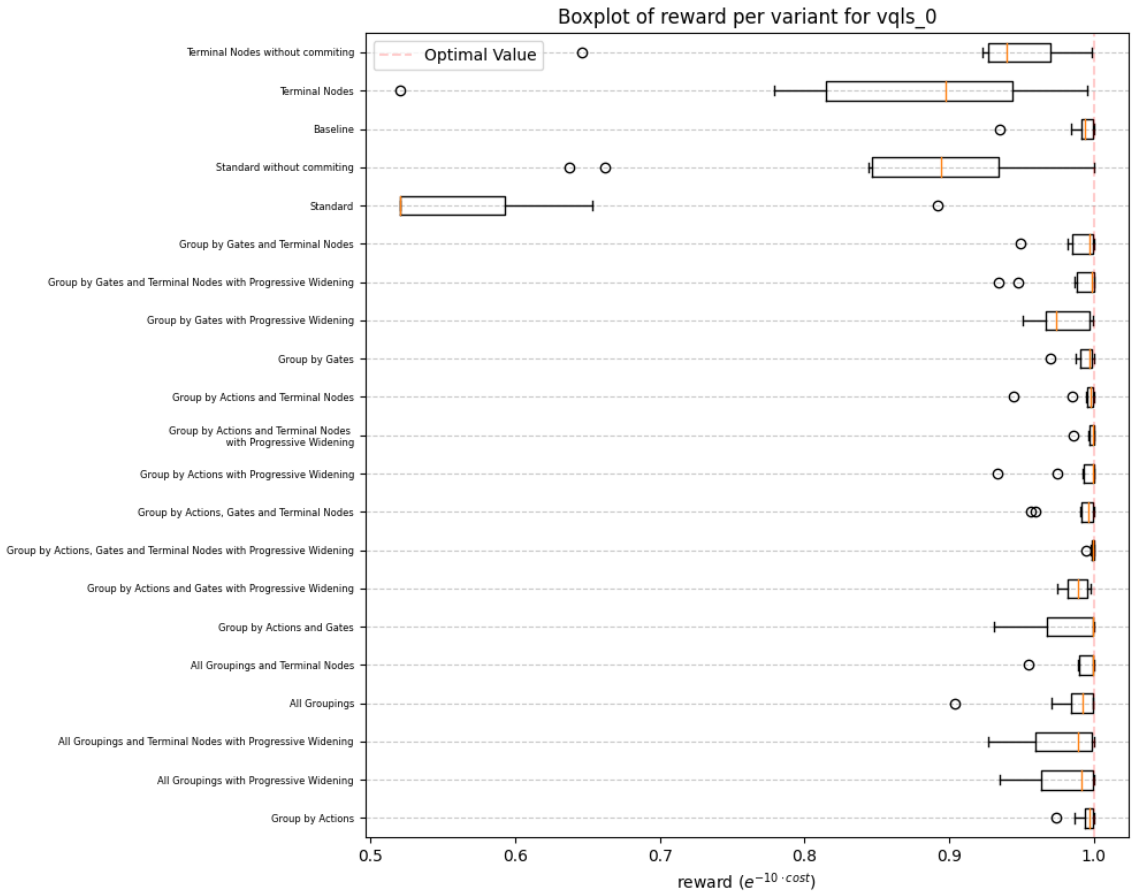

Figure B.4: All variants for problem VQLS 0

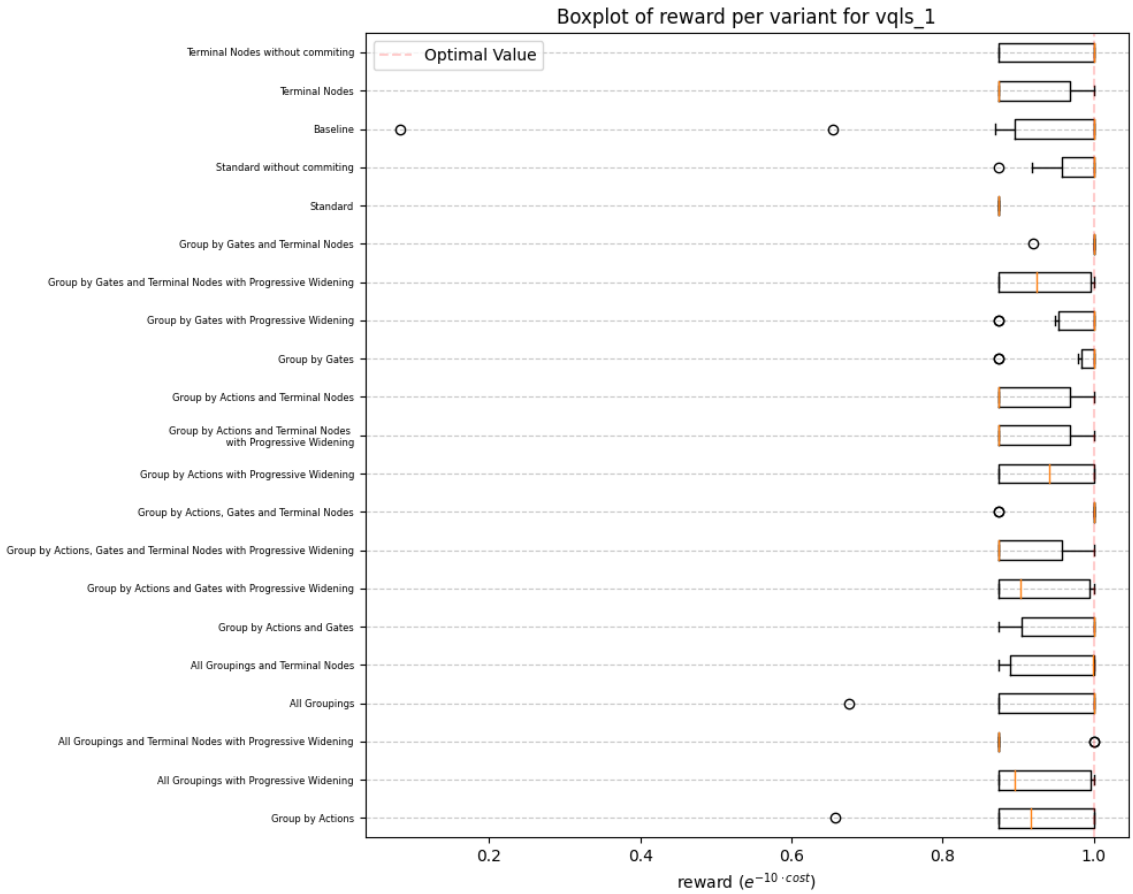

Figure B.5: All variants for problem VQLS 1

C Group effect Boxplots

The plots in this section aim to display the averaged effect that a modification such as grouping by action or terminal nodes has on a problem. The images display box plots of the highest reward values achieved after gradient descent parameter optimization for each of the 10 runs for all variants that include the modification and all variants that do not. The baseline variant's reward is plotted over just its 10 runs.

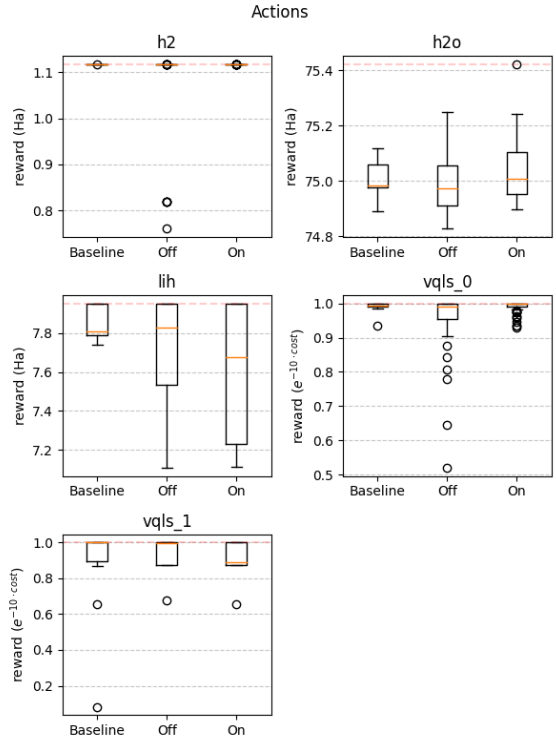

Figure C.1: Boxplots of the reward obtained by variants with grouping by action added and variants that omitted it. Plotted for all 5 problems.

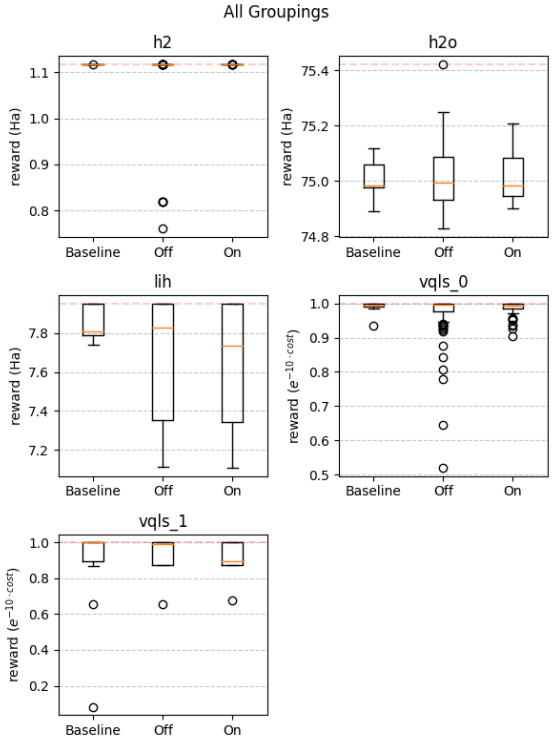

Figure C.2: Boxplots of the reward obtained by variants with all groupings added and variants that omitted it. Plotted for all 5 problems.

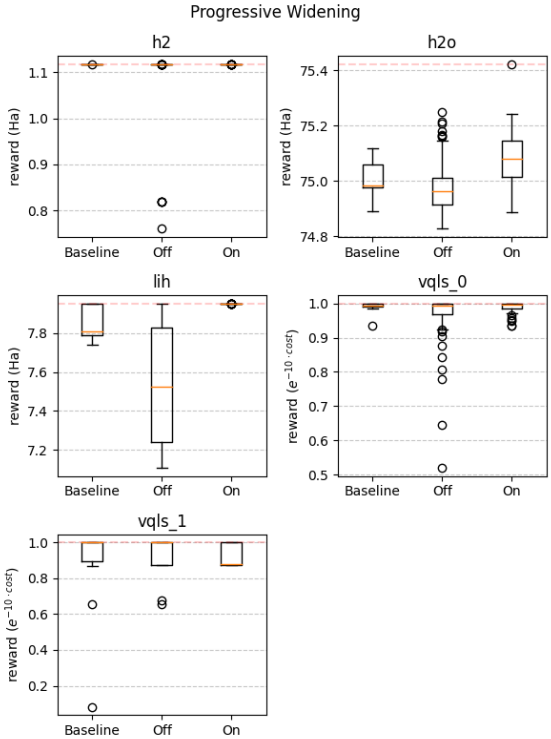

Figure C.3: Boxplots of the reward obtained by variants with progressive widening by action added and variants that omitted it. Plotted for all 5 problems.

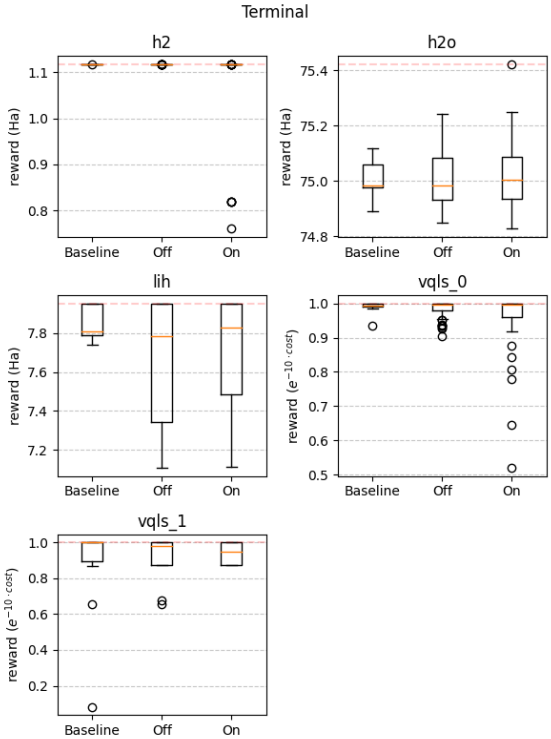

Figure C.4: Boxplots of the reward obtained by variants with terminal nodes by action added and variants that omitted it. Plotted for all 5 problems.

D Tables

problem	H_2			H_2O			LiH			VQLS 0			VQLS 1		
variant	**H**	**R**	**Cx**	**H**	**R**	**Cx**	**H**	**R**	**Cx**	**H**	**R**	**Cx**	**H**	**R**	**Cx**
Standard	2	5	2	8	8	1	10	15	3	4	4	1	4	0	2
Group by Actions	2	2	0	5	2	0	7	2	0	4	2	1	3	2	0
Group by Gates	2	2	1	6	1	0	5	1	0	4	2	1	4	3	0
Group by Actions and Gates	1	2	0	5	2	0	8	2	0	4	1	0	4	3	0
All Groupings	1	2	0	5	1	1	7	3	0	4	2	1	3	2	0
Terminal Nodes	3	8	0	8	11	2	10	16	4	4	4	0	4	4	1
Group by Actions and Terminal Nodes	1	1	1	5	2	0	8	2	0	4	1	0	4	2	1
Group by Gates and Terminal Nodes	3	5	0	6	1	0	4	0	0	4	2	1	4	3	0
Group by Actions, Gates and Terminal Nodes	2	3	0	6	2	0	8	2	0	4	1	0	4	3	0
All Groupings and Terminal Nodes	2	2	0	5	3	0	7	2	0	4	2	1	4	2	0
Standard without committing	2	4	1	8	7	2	9	12	3	4	2	0	4	4	1
Baseline	1	6	0	8	11	1	1	8	0	4	1	2	4	2	2
Group by Actions with Progressive Widening	0	5	0	4	5	1	0	2	0	4	2	2	3	2	0
Group by Gates with Progressive Widening	2	7	0	7	4	0	0	3	0	4	3	1	4	3	0
Group by Actions and Gates with Progressive Widening	2	2	0	2	5	0	0	2	0	3	2	1	4	3	0
All Groupings with Progressive Widening	2	3	0	5	3	0	0	1	0	4	1	1	4	2	1
Terminal Nodes without committing	2	5	0	8	8	1	10	15	2	4	3	2	4	3	2
Group by Actions and Terminal Nodes with Progressive Widening	1	4	1	2	6	0	0	2	0	4	1	0	4	3	0
Group by Gates and Terminal Nodes with Progressive Widening	2	6	0	6	4	0	0	2	0	4	1	1	4	3	1
Group by Actions, Gates and Terminal Nodes with Progressive Widening	1	3	0	4	3	0	0	0	0	4	3	1	4	2	0
All Groupings and Terminal Nodes with Progressive Widening	2	2	0	3	3	0	0	2	0	4	2	0	4	2	1

Table 2: Shows the gate composition for the best optimized circuit among 10 trials for diverse quantum problems for all combination of variants tested.

problem	H_2		H_2O		LiH		VQLS 0		VQLS 1	
variant	**before**	**after**	**before**	**after**	**before**	**after**	**before**	**after**	**before**	**after**
Standard	1.0200	1.1174	74.8501	74.8642	7.6120	7.6120	1.0000	1.0000	0.9211	1.0000
Group by Actions	1.0933	1.1174	75.1464	75.2381	7.9527	7.9527	1.0000	1.0000	0.9705	1.0000
Group by Gates	1.0080	1.1174	75.0167	75.1801	7.9527	7.9527	1.0000	1.0000	0.9038	1.0000
Group by Actions and Gates	1.0829	1.1174	75.2108	75.2409	7.9527	7.9527	1.0000	1.0000	0.9153	1.0000
All Groupings	1.0687	1.1174	75.2682	75.4218	7.9527	7.9527	1.0000	1.0000	0.9038	1.0000
Terminal Nodes	1.0063	1.1174	75.0122	75.2487	7.5319	7.8713	0.9956	0.9956	0.9976	1.0000
Group by Actions and Terminal Nodes	1.0838	1.1174	75.0534	75.0534	7.3163	7.3551	1.0000	1.0000	0.8161	1.0000
Group by Gates and Terminal Nodes	1.0808	1.1174	74.8598	74.8642	7.6119	7.6119	0.9998	0.9998	0.8975	1.0000
Group by Actions, Gates and Terminal Nodes	0.8537	1.1174	74.9637	75.0032	7.2910	7.3470	0.9757	1.0000	0.9757	1.0000
All Groupings and Terminal Nodes	1.1138	1.1174	74.9941	75.0568	7.9527	7.9527	0.5924	1.0000	0.9659	1.0000
Standard without commiting	1.0200	1.1174	74.9870	75.1098	7.5696	7.8713	0.9217	1.0000	0.9217	1.0000
Baseline	1.1173	1.1174	75.0802	75.1174	7.9475	7.9527	0.9512	1.0000	0.9512	1.0000
Group by Actions with Progressive Widening	1.0891	1.1174	75.1666	75.2088	7.9527	7.9527	0.9028	1.0000	0.8918	1.0000
Group by Gates with Progressive Widening	0.8980	1.1174	74.9624	75.0290	7.9527	7.9527	0.5352	1.0000	0.9299	1.0000
Group by Actions and Gates with Progressive Widening	1.0256	1.1174	75.0479	75.1456	7.9527	7.9527	0.9153	1.0000	0.9705	1.0000
All Groupings with Progressive Widening	1.0010	1.1174	74.9741	75.0246	7.9527	7.9527	0.9409	1.0000	0.9038	1.0000
Terminal Nodes without commiting	0.9791	1.1174	74.9492	75.1612	7.7194	7.9527	0.9997	1.0000	0.9997	1.0000
Group by Actions and Terminal Nodes with Progressive Widening	1.1163	1.1174	75.2682	75.4218	7.9527	7.9527	0.8918	1.0000	0.8918	1.0000
Group by Gates and Terminal Nodes with Progressive Widening	1.1041	1.1174	75.0479	75.1456	7.9527	7.9527	0.9299	1.0000	0.9299	1.0000
Group by Actions, Gates and Terminal Nodes with Progressive Widening	1.0256	1.1174	75.1366	75.2066	7.9527	7.9527	0.9705	1.0000	0.9705	1.0000
All Groupings and Terminal Nodes with Progressive Widening	1.1132	1.1174	75.1366	75.2066	7.9527	7.9527	0.5924	1.0000	0.9659	1.0000

Table 3: Shows results both for before and after circuit optimization for the best optimized circuit among 10 trials for diverse quantum problems for all combination of variants tested.